

TP 0: Mapa inicial del Backend

1) Del cliente al servidor y de HTTP a REST.

a) Explica las responsabilidades de un cliente y un servidor. Usa una acción cotidiana para describir el recorrido de una solicitud y su respuesta.

En una aplicación web, el cliente y el servidor cumplen funciones diferentes y trabajan juntos para que una aplicación funcione.

- Cliente: es el programa o dispositivo que usa la persona para hacer una solicitud. Generalmente es un navegador (Chrome, Edge, Firefox), una aplicación del celular o cualquier programa que consume un servicio.
- Servidor: es la computadora o sistema que recibe la solicitud del cliente, la procesa, accede a los datos si es necesario y envía una respuesta.

Ejemplo cotidiano: pedir una pizza

- El cliente abre la aplicación de la pizzería y elige una pizza.
- La aplicación envía el pedido al servidor.
- El servidor recibe el pedido, verifica si la pizza está disponible, calcula el precio y registra la compra.
- El servidor responde con un mensaje como: "Pedido confirmado. Llega en 30 minutos".
- El cliente muestra esa información en la pantalla.

El cliente pide información o realiza acciones, mientras que el servidor procesa esas acciones y devuelve un resultado.

b) Define API, HTTP y REST y explica porque no son sinónimos.

- API (Application Programming Interface/Interfaz de Programación de Aplicaciones)

Una API es un conjunto de reglas que permite que dos programas se comuniquen entre sí. Define qué operaciones se pueden hacer, qué datos enviar y qué respuesta esperar.

Ejemplo: la aplicación de Mercado Libre usa una API para consultar productos, precios o usuarios.

- HTTP (HyperText Transfer Protocol/Protocolo de Transferencia de Hipertexto)

HTTP es el protocolo de comunicación que usan el cliente y el servidor para enviar mensajes por Internet.

Los mensajes HTTP tienen:

- Un método (GET, POST, PUT, DELETE, etc.),
- Una dirección (URL),
- Encabezados (headers),
- Y opcionalmente un cuerpo (body).

Ejemplo: cuando abrimos una página web, el navegador hace una petición HTTP al servidor.

- REST (Representational State Transfer/Transferencia de Estado Representacional)

REST es un estilo de arquitectura para diseñar APIs sobre HTTP. Propone una serie de reglas para organizar los recursos y las operaciones de manera consistente.

En una API REST, cada recurso tiene una URL y se manipula usando los métodos HTTP adecuados.

Método	Acción REST
GET	Obtener un producto

POST	Crear un producto nuevo
PUT	Modificar un producto
DELETE	Eliminar un producto

¿Por qué no son sinónimos?

Podés combinarlos o separarlos de formas muy distintas:

- Hay APIs que no usan HTTP ni REST:

La función Math.random() de tu lenguaje de programación es una API local; no viaja por internet ni usa HTTP.

- Hay tráfico HTTP que no es una API:

Cuando abris el navegador y entrás a un sitio de noticias para leer una nota, usás HTTP para descargar texto e imágenes, pero no estás interactuando con una API de backend.

- Hay APIs que usan HTTP pero no son REST:

Una API puede usar HTTP para mandar mensajes pero romper todas las reglas de REST (por ejemplo, mandar siempre por POST a una sola ruta como /ejecutarComando en lugar de organizar recursos limpios).

- c) Investigá las restricciones del estilo REST y resumí lo que aporta cada una. Incluí un ejemplo de una API HTTP que no pueda considerarse REST.

Restricciones del estilo REST:

REST define seis restricciones principales. Si una API no respeta estas restricciones, deja de considerarse completamente REST.

Restricción	¿Qué aporta?
Cliente-Servidor	Separa la interfaz del usuario de la lógica del servidor. Esto permite que ambos evolucionen de forma independiente.
Sin estado (Stateless)	Cada solicitud debe contener toda la información necesaria. El servidor no recuerda solicitudes anteriores.
Cacheable	Las respuestas pueden indicar si pueden almacenarse en caché para mejorar el rendimiento y reducir solicitudes.
Interfaz uniforme	Todos los recursos siguen una estructura consistente de URLs, métodos HTTP y formatos de respuesta.
Sistema en capas	El cliente no necesita saber si la respuesta proviene directamente del servidor o pasa por proxies, balanceadores o cachés.
Código bajo demanda (opcional)	El servidor puede enviar código ejecutable al cliente, como JavaScript, para ampliar funcionalidades.

Ejemplo de una API HTTP que NO es REST

Tenemos una API que usa HTTP pero realiza todas las acciones mediante POST.

Solicitud	Problema
POST /obtenerUsuario	Usa POST para leer información en lugar de GET.
POST /crearUsuario	La acción aparece en la URL y no representa un recurso.
POST /eliminarUsuario	No utiliza DELETE.

¿Por qué no es REST?

- Las URLs representan acciones, no recursos. En REST las rutas deben ser sustantivos (/usuarios) y no verbos o procedimientos (/crearUsuario, /eliminarUsuario).
- No aprovecha los métodos HTTP según su significado. Ignora los verbos estándar (GET, POST, PUT, DELETE)
- Rompe la idea de una interfaz uniforme. Al no seguir las convenciones universales de diseño REST, el contrato se vuelve procedural e impredecible para los consumidores.

Versión REST del mismo ejemplo:

Método	URL
GET	/usuarios/5
POST	/usuarios
PUT	/usuarios/5
DELETE	/usuarios/5

En este caso, usuarios es el recurso y el método HTTP indica la operación.

2) Principios SOLID en un Backend.

a) Escribí el nombre completo de cada principio de SOLID, explicarlo con palabras propias y acompañalo con un ejemplo breve de backend.

- **SRP — Single Responsibility Principle (Principio de Responsabilidad Única)**
Una clase o módulo debe tener una única razón para cambiar. Esto significa que debe encargarse de un único actor o aspecto funcional del negocio, evitando convertirse en una "clase dios" que mezcle lógica no relacionada.
Ejemplo en Backend: En lugar de que la clase Usuario guarde los datos en la base de datos, valide contraseñas y envíe emails de bienvenida, se separan las tareas: un UsuarioRepository (persistencia), un UsuarioValidator (validación de datos) y un EmailService (notificaciones).
- **OCP — Open/Closed Principle (Principio de Abierto/Cerrado)**
Las entidades de software deben estar abiertas para su extensión, pero cerradas para su modificación. Si queremos agregar una nueva funcionalidad, deberíamos poder hacerlo creando nuevo código (clases/estrategias) sin alterar el código existente que ya fue probado.
Ejemplo en Backend: Un sistema de cobros tiene un servicio ProcesadorPagos. En lugar de tener un switch o múltiples if para cada

pasarela de pago (if (metodo == "MercadoPago") ... else if (metodo == "Stripe")), se crea una interfaz MetodoPago con el método pagar(). Al añadir *PayPal*, solo se crea una nueva clase PayPalPago sin modificar la clase ProcesadorPagos.

- **LSP — Liskov Substitution Principle (Principio de Sustitución de Liskov)**
Las subclases o clases derivadas deben poder sustituir a sus clases base o interfaces sin alterar el comportamiento correcto del programa. Una subclase no debe romper contratos, lanzar excepciones inesperadas o anular métodos vaciándolos.

Ejemplo en Backend: Si existe una clase base BaseDeDatos con el método guardar(), y creamos una subclase BaseDeDatosLectura que lanza un error OperacionNoSoportadaException al llamar a guardar(), estamos violando Liskov. La solución es segregar las interfaces en LectorDatos y EscritorDatos.

- **ISP — Interface Segregation Principle (Principio de Segregación de Interfaces)**

Ningún cliente debería verse forzado a depender de métodos que no utiliza. Es preferible tener muchas interfaces pequeñas y específicas en lugar de una interfaz "monolítica" o sobrecargada.

Ejemplo en Backend: En vez de tener una interfaz gigante IControladorArchivos con subirVideo(), subirImagen() y transcodificarAudio(), se divide en interfaces específicas como ISubidorImágenes y ITranscodificador. Un servicio de avatares de perfil solo implementará ISubidorImágenes.

- **DIP — Dependency Inversion Principle (Principio de Inversión de Dependencias)**

Los módulos de alto nivel (lógica de negocio o casos de uso) no deben depender de módulos de bajo nivel (infraestructura, bases de datos, APIs externas); ambos deben depender de abstracciones (interfaces). A su vez, las abstracciones no deben depender de los detalles, sino los detalles de las abstracciones.

Ejemplo en Backend: Un caso de uso CrearPedidoUseCase no debe instanciar directamente un new MySQLPedidoRepository(), sino depender de una interfaz IPedidoRepository. Luego, mediante inyección de dependencias, se le pasa la implementación concreta (MySQL, PostgreSQL o un Mock para tests unitarios).

b) Una clase válida solicitudes, aplica reglas de negocio, consulta la base de datos y envía correos. Explicá que principios permitirían mejorarla y por qué aplicar SOLID no significa crear abstracciones para todo.

El principal problema es que esta clase tiene demasiadas responsabilidades. El principio que se ve afectado más claramente es SRP, porque existen varias razones diferentes por las que la clase podría necesitar ser modificada.

Por ejemplo, podría cambiar una regla de negocio, cambiar la estructura de la base de datos o cambiar el proveedor utilizado para enviar correos, y todos esos cambios terminarían afectando a la misma clase.

Para mejorarla:

→ Principio de Responsabilidad Única (SRP):

- ◆ La clase actual tiene al menos 4 razones distintas para cambiar (cambios en el formato de entrada, cambios en las reglas del negocio, cambios en el motor de base de datos o cambios en el proveedor de emails).

Se descompone la clase delegando responsabilidades en componentes especializados:

- >Validador / DTO: Valida la estructura y formato de la petición.
- >CasoDeUso / Servicio de Dominio: Aplica las reglas del negocio.
- >Repositorio: Consulta y persiste los datos.
- >ServicioDeNotificacion: Gestiona el envío de correos.

→ Principio de Inversión de Dependencias (DIP):

- ◆ La lógica de negocio está fuertemente acoplada a implementaciones técnicas de bajo nivel (un driver concreto de BD o una librería de correo específica).
- >La lógica de negocio pasa a depender de interfaces abstractas (ej. IRepositorio y IEmailService). Esto facilita cambiar de proveedor (por ejemplo, pasar de SendGrid a Amazon SES) sin tocar las reglas de negocio y permite realizar pruebas unitarias con dobles de prueba (mocks).

→ Principio de Segregación de Interfaces (ISP):

- ◆ Si para acceder a estas tareas se creará una interfaz monolítica, los consumidores dependerían de operaciones innecesarias.
- >Se definen contratos pequeños y focalizados para la persistencia y la mensajería.

¿Por qué aplicar SOLID NO significa crear abstracciones para todo?

Aplicar SOLID de forma indiscriminada puede llevar al **sobre-diseño (over-engineering)**, introduciendo capas innecesarias de complejidad:

Si una clase tiene una responsabilidad simple, no cambia y no genera un problema de mantenimiento, separarla artificialmente puede hacer que el proyecto sea más complicado en lugar de mejorarlo. Por ejemplo, crear cinco clases diferentes solamente para realizar una operación muy sencilla puede agregar código innecesario y dificultar la comprensión del programa. SOLID debe utilizarse como una guía para detectar problemas de diseño, especialmente cuando el sistema crece o cuando existen responsabilidades demasiado mezcladas, y no como una obligación de llenar el proyecto de abstracciones.

SOLID busca que cada parte del backend tenga responsabilidades claras, que las dependencias sean manejables y que el sistema pueda evolucionar sin tener que modificar grandes cantidades de código cada vez que aparece un cambio. La clave no es aplicar los cinco principios de manera automática, sino utilizarlos cuando realmente ayudan a que el código sea más simple de mantener y extender.

3) El lenguaje de HTTP

a) Investiga GET, HEAD, POST, PUT, PATCH, DELETE, CONNECT, OPTIONS y TRACE. Resumí en una tabla su finalidad, un ejemplo y si son seguros e idempotentes. Aclará también que existen métodos de extensión.

- Seguro: La petición es de solo lectura. No altera ni destruye datos en el servidor.
- Idempotente: Ejecutar la misma petición 1 vez o 100 veces seguidas deja el servidor exactamente en el mismo estado final.

Método	Para qué Sirve	Ejemplo de Uso	¿Es seguro?	¿Es idempotente?
GET	Pedir y consultar información sin modificar nada.	GET /usuarios/5 (Ver el usuario 5)	Si	Si

HEAD	Igual que GET, pero solo pide los encabezados (headers), sin el cuerpo/contenido. Sirve para chequear si un archivo cambió o su peso antes de descargarlo.	HEAD /archivo.zip	Si	Si
POST	Enviar datos para crear un recurso nuevo o procesar información.	POST /usuarios (Crear un nuevo usuario con los datos enviados)	No	No
PUT	Reemplazar o crear por completo un recurso existente con los datos enviados.	PUT /usuarios/5 (Sobrescribe todo el perfil del usuario 5)	No	Si
PATCH	Modificar parcialmente un recurso (cambiar solo uno o dos campos).	PATCH /usuarios/5 (Cambiar únicamente el email del usuario 5)	No	No
DELETE	Eliminar un recurso del servidor.	DELETE /usuarios/5 (Borrar al usuario 5)	No	Si
CONNECT	Establece un túnel de comunicación bidireccional (usado para conexiones seguras HTTPS a través de un proxy).	CONNECT api.banco.com:443	No	No
OPTIONS	Preguntar al servidor qué métodos o permisos tiene permitidos una ruta (muy usado por los navegadores en CORS).	OPTIONS /usuarios (¿Puedo hacer DELETE aquí?)	Si	Si
TRACE	Hace un "eco" del mensaje para diagnóstico y pruebas de red (saber si un proxy modificó la petición).	TRACE /api/ping	Si	Si

Métodos de Extensión:

HTTP no se limita solo a estos verbos estándar. El protocolo permite definir métodos personalizados (métodos de extensión) para necesidades específicas. Un ejemplo real es el protocolo **WebDAV**, que añade métodos como COPY, MOVE, LOCK o MKCOL para gestionar archivos remotos.

b) ¿Se debería usar GET para modificar información en una base de datos? Explicá por qué considerando caché, reintentos, prefetching y rastreadores automáticos.

Usar GET para acciones como borrar o editar datos (por ejemplo, GET /usuarios/5/borrar) es una pésima práctica por cuatro razones críticas:

- **Caché (Navegadores y Proxies):** Como GET se considera una lectura segura, los navegadores y servidores intermedios guardan la respuesta en memoria. Si intentas ejecutar la acción otra vez, el navegador podría responder con la copia guardada sin avisar al servidor, haciendo que la modificación nunca ocurra.
- **Prefetching (Precarga del navegador):** Para acelerar la navegación, muchos exploradores modernos descargan de fondo los enlaces (<a>) que ven en una página antes de que el usuario haga click. Si un enlace de borrado usa GET, el navegador podría borrar registros automáticamente sin que el usuario lo haya solicitado.
- **Rastreadores automáticos (Web Crawlers / Bots):** Los bots de motores de búsqueda (como Googlebot) recorren e indexan todos los enlaces que encuentran mediante peticiones GET. Si las rutas de modificación usan GET, el bot podría vaciar tu base de datos simplemente recorriendo tu sitio.
- **Reintentos automáticos de red:** Si una conexión a internet falla o se corta, los clientes HTTP y navegadores reintentan las peticiones GET de forma automática porque asumen que no causan efectos secundarios. Si modificara datos, podrías duplicar compras o acciones sin querer.

c) Explica las familias 1xx a 5xx y el uso de los códigos más comunes: 200, 201, 204, 400, 401, 403, 404, 409, 422, 429, 500 y 503.

Las respuestas HTTP se agrupan en 5 familias según el primer dígito:

- **1xx (Informativos):** La petición fue recibida y el proceso continúa.
- **2xx (Éxito):** La petición fue recibida, entendida y aceptada correctamente.
- **3xx (Redirecciones):** El cliente debe tomar una acción adicional para completar la solicitud (ej. el recurso cambió de lugar).
- **4xx (Errores del Cliente):** Hubo un error en la solicitud enviada por quien consume la API (datos inválidos, falta de permisos, ruta inexistente).
- **5xx (Errores del Servidor):** El cliente envió todo bien, pero el servidor falló internamente al intentar procesarla.

Códigos más comunes:

- **200 OK:** Todo salió bien. Se usa para devolver datos en consultas (GET) o confirmaciones de operaciones (PUT/PATCH).
- **201 Created:** El recurso se creó con éxito. Es la respuesta estándar para un POST que genera un nuevo registro.
- **204 No Content:** La acción se ejecutó con éxito, pero la respuesta no necesita devolver ningún contenido (muy común al hacer un DELETE).
- **400 Bad Request:** Solicitud mal armada (sintaxis rota, JSON mal escrito o datos que el servidor no puede interpretar).
- **401 Unauthorized:** Falta autenticación. El usuario no ha iniciado sesión o no envió un token de acceso válido.
- **403 Forbidden:** El servidor sabe quién sos (estás autenticado), pero **no tenés permisos** suficientes para realizar esa acción.
- **404 Not Found:** El recurso o la URL solicitada no existe en el servidor.
- **409 Conflict:** La petición entra en conflicto con el estado actual del sistema (por ejemplo, intentar registrar un usuario con un email que ya existe).

- **422 Unprocessable Entity:** El formato del mensaje es correcto, pero los datos no pasan las validaciones de negocio (ejemplo: un campo de edad contiene un número negativo o una fecha de nacimiento futura).
- **429 Too Many Requests:** El cliente superó el límite de peticiones permitidas en un período de tiempo (activación de *Rate Limiting*).
- **500 Internal Server Error:** Ocurrió un error no controlado en el backend (un bug en el código, excepción no atrapada o fallo de base de datos).
- **503 Service Unavailable:** El servidor no puede atender la solicitud temporalmente porque está sobrecargado o en mantenimiento.

4) Estado, identidad y permisos

- a) **Comparará aplicaciones stateless y stateful: donde conservan el estado, ventajas, costos, y un ejemplo de cada una. Aclarará si un backend stateless puede consultar una base de datos.**

Criterio	Stateful (Con estado)	Stateless (Sin estado)
¿Dónde se guarda el estado?	En la memoria del servidor donde el usuario interactúa (sesiones de servidor).	En el cliente (mediante tokens como JWT) o en un almacén de datos centralizado (Base de Datos / Redis).
Ventajas	• Fácil de implementar en aplicaciones monolíticas pequeñas. • El servidor tiene control total e instantáneo para revocar una sesión en su propia memoria.	• Escalabilidad horizontal sencilla: cualquier servidor puede atender cualquier petición sin sincronizar memoria. • Mayor tolerancia a fallos: si un servidor se cae, otro toma la petición sin problemas.
Costos y desventajas	• Difícil escalamiento horizontal: requiere sesiones persistentes (<i>sticky sessions</i>) o sincronización compleja entre réplicas. • Si un servidor se reinicia, todos los usuarios conectados pierden su sesión.	• Cada petición debe viajar con todo el contexto/token, consumiendo algo más de ancho de banda. • La revocación inmediata de credenciales requiere mecanismos adicionales.
Ejemplo cotidiano	Una partida de ajedrez online por WebSockets o un carrito de compras viejo guardado solo en la memoria RAM del servidor.	Una API REST para una aplicación móvil donde cada petición incluye un token JWT en el header Authorization.

¿Un backend Stateless puede consultar una base de datos?

Un backend stateless puede perfectamente consultar una base de datos. La diferencia es que no mantiene el estado de la sesión o de la interacción del cliente en la memoria de la instancia del servidor entre solicitudes.

b) Diferenciá autenticación y autorización mediante un ejemplo con roles y permisos.

- Autenticación (¿Quién sos?) Es el proceso de validar la identidad de un usuario (demostrar que alguien es quien dice ser mediante email y contraseña, biometría o un token).
- Autorización (¿Qué tenés permitido hacer?) Es el proceso de comprobar si un usuario ya identificado tiene los permisos y roles necesarios para realizar una acción específica sobre un recurso.

→ Por ejemplo: un sistema de administración de una tienda.

- ◆ Lucía inicia sesión utilizando su usuario y contraseña. El servidor verifica las credenciales y comprueba que realmente es Lucía. Eso es autenticación.
- ◆ Después, el sistema determina que Lucía tiene el rol administrador. Ese rol tiene permisos como:
 - Crear productos.
 - Modificar productos.
 - Eliminar productos.
 - Consultar ventas.

Si Lucía intenta acceder a:

DELETE /productos/25

El servidor primero necesita saber quién es —**autenticación**— y después comprobar si tiene permiso para eliminar productos —**autorización**.

- ◆ En cambio, si otro usuario tiene el rol cliente, podría tener permisos para:
 - Consultar productos.
 - Realizar compras.
 - Consultar sus propios pedidos.

Pero no tendría permiso para eliminar productos.

c) Explicá las partes de un JWT, que protege la firma y porque Base64URL no cifra su contenido. Indicá que debería validar el servidor y una dificultad de revocar estos tokens.

Un JWT (JSON Web Token) es un formato utilizado para transmitir información entre partes, habitualmente para representar información sobre la identidad y los permisos de un usuario.

Las 3 partes de un JWT:

- Header (Encabezado): Contiene el tipo de token (JWT) y el algoritmo criptográfico utilizado para la firma (por ejemplo, HS256 o RS256).
- Payload (Carga útil / Claims): Contiene los datos o declaraciones sobre el usuario (ID de usuario, rol, fecha de expiración exp).
- Signature (Firma): Se genera combinando el Header codificado, el Payload codificado y una clave secreta del backend mediante el algoritmo elegido.

¿Qué protege la firma y por qué Base64URL no cifra?

- **Qué protege la firma:** Garantiza la integridad y la autenticidad. Si alguien altera un solo dato del payload (por ejemplo, cambia "rol": "user" por "rol": "admin"), la firma ya no coincidirá y el servidor rechazará el token.
- **Base64URL no es cifrado:** Base64URL es únicamente un formato de codificación para que el texto viaje seguro en URLs y cabeceras HTTP sin romper caracteres. No oculta la información: cualquiera que intercepte un JWT puede decodificar el payload en segundos y leer su contenido. Por eso nunca deben guardarse datos sensibles (como contraseñas o números de tarjetas) en el payload.

¿Qué debe validar el servidor al recibir un JWT?

- **La firma criptográfica:** Comprobar con su clave secreta que el token fue emitido por el backend y no fue adulterado.
- **La expiración (exp):** Verificar que la fecha y hora actual no hayan superado el tiempo de vida del token.
- **El emisor y audiencia (iss, aud) si aplican:** Validar que el token haya sido generado por el servicio correcto y esté dirigido a la API actual.

Dificultad principal para revocar un JWT:

→ Como los JWT son autónomos (*stateless*) y el servidor no guarda un registro de sesiones activas en memoria, no se puede "desloguear" o invalidar un token de forma inmediata antes de que alcance su fecha de expiración natural. Si un atacante roba un token válido por 2 horas, tendrá acceso durante esas 2 horas a menos que el backend implemente una lista de bloqueo (*blacklist*) en memoria rápida (como Redis), lo cual reintroduce estado al sistema.

5) Control de tráfico y reintentos seguros

a) Explicá qué problema resuelve el rate limiting e investigá al menos dos estrategias para aplicarlo.

El Rate Limiting (límite de tasa de peticiones) es un mecanismo de defensa que restringe el número de solicitudes que un cliente puede hacer a un servidor dentro de una ventana de tiempo determinada.

Resuelve tres problemas críticos:

- Denegación de Servicio (DoS/DDoS) y saturación:** Evita que clientes maliciosos agoten los recursos del servidor (CPU, memoria, conexiones a la base de datos).
- Abuso y ataques de fuerza bruta:** Frena ataques automatizados para adivinar contraseñas (/login) o raspar datos (*web scraping*).
- Control de costos en APIs de terceros:** Protege al backend de superar cuotas pagas en servicios externos (ej. pasarelas de pago o APIs de mensajería).

Dos estrategias para aplicarlo:

- Ventana Fija (*Fixed Window Counter*):**
 - Divide el tiempo en bloques fijos (por ejemplo, bloques de 1 minuto como de 12:00 a 12:01). Cada petición en esa ventana incrementa un contador. Si llega a 100, se bloquea hasta que inicie el siguiente bloque (12:01), reseteando el contador a cero.
 - Muy simple de implementar y consume poca memoria.
 - Sufre del problema de "borde de ventana": un cliente puede mandar 100 peticiones a las 12:00:59 y 100 más a las 12:01:01 (200 peticiones en 2 segundos) sin superar el límite de ninguna ventana individual.
 - Balde con Fuga (*Leaky Bucket*):**
 - Las solicitudes entrantes entran en una cola (el "balde") de capacidad fija. El servidor procesa las peticiones de esa cola a una velocidad constante y uniforme (el "goteo"). Si el balde se llena, las peticiones sobrantes se descartan inmediatamente con un error 429.
 - Suaviza los picos de tráfico y garantiza que el backend nunca reciba ráfagas descontroladas.
- b) Implementa, en cualquier lenguaje o pseudocódigo, un límite de 100 solicitudes por minuto usando la IP del usuario. Incluí el contador con vencimiento, la respuesta 429 y explicá sus límites al ejecutar varias instancias o compartir una IP.

```

CONSTANTE LIMITE_MAXIMO = 100
CONSTANTE DURACION_VENTANA = 60 // Segundos

// Estructura en memoria: IP -> { solicitudes: entero, vencimiento: marca_de_tiempo }
contadorIPs = mapa vacío

funcion middlewareRateLimit(solicitud, respuesta):
    ip = solicitud.obtenerIP()
    ahora = tiempoActualEnSegundos()

    // 1. Si no existe registro o la ventana anterior ya expiró, reiniciamos el ciclo
    si (ip no existe en contadorIPs) 0 (ahora >= contadorIPs[ip].vencimiento):
        contadorIPs[ip] = {
            solicitudes: 1,
            vencimiento: ahora + DURACION_VENTANA
        }
        permitir solicitud
        retornar

    // 2. Si ya superó el cupo dentro de la ventana de tiempo activa
    si contadorIPs[ip].solicitudes >= LIMITE_MAXIMO:
        tiempoRestante = contadorIPs[ip].vencimiento - ahora

        respuesta.establecerCabecera("Retry-After", tiempoRestante)
        respuesta.enviarEstado(429, {
            "error": "Too Many Requests",
            "mensaje": "Has superado el límite de 100 solicitudes por minuto.",
            "reintentar_en_segundos": tiempoRestante
        })
        detener solicitud
        retornar

    // 3. Si aún tiene cupo disponible, incrementamos y permitimos el paso
    contadorIPs[ip].solicitudes = contadorIPs[ip].solicitudes + 1
    permitir solicitud
fin funcion

```

c) **Defini idempotencia, analizá como se comportan GET, PUT, DELETE, POST y PATCH y explica el uso de Idempotency-Key al reintentar un pago.**

>La idempotencia es la propiedad por la cual ejecutar una misma solicitud una o múltiples veces idénticas produce exactamente el mismo estado final en el servidor y el mismo efecto secundario que haberla ejecutado una sola vez.

Ejemplo: borrar un recurso dos veces no lo “borra doble”, simplemente sigue sin existir.

Método	¿Es idempotente?	Justificación
GET	Si	Es de solo lectura. Consultar /articulos/10 1 vez o 50 veces no modifica ningún dato en el backend.
PUT	Si	Reemplaza un recurso por completo. Enviar el mismo payload a PUT /articulos/10 repetidas veces deja el recurso con el mismo contenido

		exacto.
DELETE	Si	La primera llamada elimina el recurso; las siguientes llamadas encuentran que ya no existe. El estado final del sistema es el mismo: el recurso no está.
POST	No	Se utiliza para crear recursos o iniciar procesos. Repetir POST /ordenes enviará múltiples compras y generará múltiples cobros.
PATCH	No	Aunque modificar un campo fijo (PATCH /usuarios/1 con {"nombre": "Ana"}) actúa de forma idempotente, una instrucción incremental (PATCH /saldo con {"incrementar": 50}) alterará el estado en cada ejecución.

Idempotency-Key en pagos:

Un POST para crear un pago puede fallar en la red. El cliente reintenta y el servidor podría procesar dos cobros.

- **Solución:**
 - El cliente genera un **UUID único** como *Idempotency-Key*.
 - Lo envía en la cabecera de cada reintento.
 - El servidor guarda la clave junto con el resultado del primer procesamiento.
 - Si recibe otra petición con la misma clave y el mismo payload, devuelve la respuesta original sin ejecutar la lógica de negocio otra vez.

6) Contratos que crecen sin romper clientes

a) Explicá la paginación por página, offset y cursor. Sin implementar código, escribí un ejemplo de contrato JSON paginado para GET /api/productos, con sus parámetros y metadatos.

- **Por página (page y limit):** Se indica el número de página y el tamaño.
Ejemplo: ?page=2&limit=20. Fácil de usar, pero sensible a inserciones/borrados (los resultados pueden "moverse").
- **Por offset (offset y limit):** Se indica el desplazamiento y el tamaño.
Ejemplo: ?offset=40&limit=20. Similar a página, pero más flexible. Problema: ineficiente en bases grandes (requiere saltar registros).
- **Por cursor (cursor / after y limit):** Se usa un identificador único del último elemento recibido.
Ejemplo: ?cursor=abc123&limit=20. Escalable y consistente, ideal para grandes volúmenes o datos en tiempo real.

Solicitud: GET /api/productos?page=2&limit=2

Respuesta JSON (200 OK):

```
{
  "data": [
    {
      "id": 3,
      "nombre": "Teclado Mecánico RGB",
      "precio": 85.50,
      "categoria": "Periféricos",
      "activo": true
    },
    {
      "id": 4,
      "nombre": "Mouse Inalámbrico",
      "precio": 45.00,
      "categoria": "Periféricos",
      "activo": true
    }
  ],
  "pagination": {
    "page": 2,
    "limit": 2,
    "total_items": 45,
    "total_pages": 23,
    "has_next": true,
    "has_prev": true,
    "next_page_url": "/api/productos?page=3&limit=2",
    "prev_page_url": "/api/productos?page=1&limit=2"
  }
}
```

- b) **Investigá las formas habituales de versionar una API y explicá cuando un cambio obliga a crear una nueva versión.**

Formas habituales de versionar una API:

Estrategia	Ejemplo de uso	Ventaja
En la URI / Ruta	https://api.ejemplo.com/v1/productos	Muy visible, fácil de probar y simple de enrutar a nivel de proxies.
Por Query Params	https://api.ejemplo.com/productos?v=1	Mantiene una URI base fija, simple para pruebas rápidas.
Por Cabeceras Personalizadas	X-API-Version: 1	Mantiene las URIs limpias y semánticas.
Por Negociación de Contenido (<i>Header Accept</i>)	Accept: application/vnd.ejemplo.v1+json	Estándar RESTful más puro; asocia la versión directamente a la representación del recurso.

¿Cuándo un cambio obliga a crear una nueva versión?

Solo se debe versionar cuando ocurre un cambio de ruptura (*breaking change*) que rompe los contratos con clientes existentes:

Requiere nueva versión:

- Renombrar o eliminar un campo existente en la respuesta JSON.
- Cambiar el tipo de dato de un campo (por ejemplo, transformar un id: 123 numérico a id: "usr_123" en texto).
- Convertir un parámetro opcional en obligatorio.
- Cambiar la estructura jerárquica de la respuesta (ej. de un arreglo plano a un objeto anidado).

c) **Definí origen, CORS y preflight. Indica que permitiría comunicar un frontend en <https://app.ejemplo.com> con una API en <https://api.ejemplo.com> y porque CORS no reemplaza la autenticación.**

- **Origen:** Un origen web está definido por la combinación exacta de tres componentes: Protocolo (Esquema) + Dominio (Host) + Puerto. Si alguno de los tres difiere, se considera un origen cruzado (*Cross-Origin*).
- **CORS (*Cross-Origin Resource Sharing*):** Es un mecanismo de seguridad implementado por los navegadores web (basado en la *Same-Origin Policy*) que restringe que una página web cargada desde un origen haga peticiones AJAX a una API ubicada en un origen diferente, a menos que el servidor dé su consentimiento explícito mediante cabeceras HTTP.
- **Petición Preflight:** Es una solicitud previa automática de verificación que realiza el navegador utilizando el método OPTIONS antes de enviar la petición real (cuando la petición usa métodos como PUT, DELETE o cabeceras personalizadas como Authorization). El navegador consulta si el servidor permite esa operación y qué cabeceras acepta.

¿Se pueden comunicar directamente?

Un navegador web considera que dos direcciones son orígenes distintos si cambia el protocolo, el dominio o el puerto:

- **Frontend:** <https://app.ejemplo.com> (Subdominio: **app**)
- **Backend/API:** <https://api.ejemplo.com> (Subdominio: **api**)
Aunque ambos terminen en ejemplo.com, para el navegador son dos sitios diferentes. Por seguridad, el navegador bloquea automáticamente que el frontend lea datos de esa API.

¿Por qué CORS NO reemplaza a la autenticación?

- **CORS es una protección del navegador para el usuario, no una defensa de la API:** La política CORS solo la ejecutan los navegadores para evitar que un sitio malicioso use las cookies o sesiones activas del usuario para consultar APIs ajenas en su nombre.
- **CORS no protege contra llamadas directas:** Cualquier herramienta fuera del navegador (como curl, Postman, scripts en Python o un backend atacante) **ignora por completo CORS** y puede disparar peticiones directas al servidor.
- **La autenticación/autorización es obligatoria en el backend:** Solo los tokens (JWT), claves de API y la validación de permisos en el servidor garantizan realmente la identidad y el acceso seguro a los datos.

7) **Elegir un modelo de datos**

a) **Compará bases SQL y NoSQL según estructura, relaciones, consultas, transacciones, consistencia y escalabilidad.**

Criterio	SQL (Relacionales)	NoSQL (No relacionales)
Estructura	Rígida y tabular: Los datos se guardan en tablas con filas y columnas bajo un esquema fijo (<i>schema</i>).	Flexible y dinámica: Los datos se guardan en documentos (JSON), grafos o clave-valor sin un esquema estricto.
Relaciones	Nativas y fuertes: Une datos de distintas tablas mediante claves foráneas y uniones (JOIN).	Débiles o inexistentes: Los datos relacionados suelen guardarse juntos dentro del mismo documento (anidados).
Consultas	Lenguaje estandarizado (SQL): Permite consultas complejas, filtros avanzados y agregaciones con sintaxis común.	APIs y sintaxis específicas: Varía según el motor (ej. consultas de MongoDB o comandos en Redis).
Transacciones	Garantías ACID estrictas: Asegura que una operación compleja se cumpla al 100% o no se guarde nada.	Modelo BASE: Prioriza la velocidad y disponibilidad antes que la consistencia inmediata.
Consistencia	Inmediata y fuerte: Si un dato se actualiza, cualquier usuario que consulte un milisegundo después ve el cambio.	Eventual: Puede tardar unos segundos en propagarse a todas las copias o servidores.
Escalabilidad	Vertical: Se mejora la potencia del mismo servidor (más memoria RAM, mejor procesador o disco).	Horizontal: Se añaden más servidores económicos en red (<i>sharding</i> / <i>clúster</i>).

b) Indica qué enfoque elegirías para un sistema universitario, un registro masivo de eventos y un catálogo con atributos variables. Justifica y explicá si una aplicación puede combinar ambos enfoques.

- **Sistema universitario (Gestión académica):**
 - SQL (ej. PostgreSQL o MySQL).
 - Los datos tienen relaciones complejas y estrictas: alumnos, materias, correlatividades, actas de examen y pagos. Es crítico contar con integridad referencial (no podés anotar a un alumno inexistente en una materia) y transacciones ACID (aprobar un examen y actualizar el promedio debe ser atómico y consistente de inmediato).
- **Registro masivo de eventos (Logs, telemetría de sensores IoT):**
 - NoSQL
 - Se trata de un volumen gigantesco de datos de solo escritura a una velocidad altísima. No se necesitan relaciones complejas entre eventos pasados, sino la capacidad de escribir miles de registros por segundo y escalar horizontalmente añadiendo nuevos servidores sin saturar el sistema.
- **Catálogo de productos con atributos variables:**
 - NoSQL
 - Cada categoría de producto tiene propiedades completamente distintas: una remera tiene *talle*, *color* y *tela*; una computadora tiene *memoria RAM*, *procesador* y *almacenamiento*. Un modelo de

documentos JSON permite que cada producto guarde sus propios campos libremente sin obligar a crear decenas de tablas intermedias vacías en SQL.

¿Una aplicación puede combinar ambos enfoques?

Sí, y es la práctica estándar en la industria. Este patrón se conoce como Persistencia Políglota (*Polyglot Persistence*).

Consiste en utilizar el motor de base de datos más adecuado para cada necesidad puntual dentro del mismo sistema:

- **SQL (PostgreSQL):** Gestiona los usuarios, cobros, facturación y pedidos (donde la consistencia del dinero y las cuentas es crítica).
- **NoSQL de Documentos (MongoDB):** Almacena el catálogo de productos con atributos dinámicos.
- **NoSQL Clave-Valor (Redis):** Maneja la caché rápida y el rate limiting en memoria para no sobrecargar las bases principales.
- **NoSQL de Búsqueda (Elasticsearch):** Procesa búsquedas de texto predictivas y filtros rápidos de productos.

8) Organizar y encontrar datos

a) **Explicá que es un índice, como acelerar las consultas y que costos introduce. Incluí los tipos de índices más habituales y un ejemplo de cuándo conviene usar uno.**

- **Índices en Bases de Datos:**
 - Es una estructura de datos auxiliar (similar al índice alfabético al final de un libro) que guarda una lista ordenada de los valores de una o varias columnas junto con un puntero a la ubicación física de la fila en el disco.
 - ¿Cómo acelera las consultas?
Sin un índice, el motor debe realizar un escaneo secuencial completo (Full Table Scan), leyendo fila por fila desde la primera hasta la última. Con un índice, el motor realiza una búsqueda binaria o por árbol, saltando directamente a los registros buscados en una fracción de milisegundo.
 - Costos que introduce:
 - **Espacio en disco y memoria RAM:** Cada índice es una copia estructurada de datos que ocupa almacenamiento adicional.
 - **Penalización en escrituras (INSERT, UPDATE, DELETE):** Cada vez que se agrega, modifica o borra un registro, la base de datos debe actualizar tanto la tabla como todos los índices asociados, ralentizando las operaciones de escritura.
- **Tipos de índices más habituales:**
 - **B-Tree (Árbol B):** El índice estándar por defecto. Ideal para búsquedas exactas (=), rangos (BETWEEN, >, <) y ordenamientos (ORDER BY).
 - **Hash:** Estructura basada en tablas hash. Extremadamente rápido para comparaciones de igualdad exacta (=), pero no sirve para rangos ni ordenamientos.
 - **GIN / GiST (Índices invertidos):** Usados para tipos de datos complejos como texto completo (*Full-Text Search*), arreglos o datos JSON/geométricos.
- **Ejemplo de uso:**
En una tabla de usuarios con 5 millones de filas, las personas inician sesión buscando por su correo (`SELECT * FROM usuarios WHERE email =`

'juan@mail.com'). Crear un índice **B-Tree sobre la columna email** evita revisar los 5 millones de registros y encuentra al usuario en solo 3 o 4 lecturas de disco.

b) Definí la primera, segunda y tercera forma normal y BCNF. Explica qué problemas evita la normalización y cuando podría aceptarse una desnormalización controlada.

- **1FN (Primera Forma Normal):**
Cada celda debe contener un único valor indivisible (atómico) y no deben existir grupos repetidos ni listas dentro de un solo campo.
- **2FN (Segunda Forma Normal):**
Debe cumplir la 1FN y todos los atributos que no formen parte de la clave primaria deben depender totalmente de la clave primaria completa, no solo de una parte de ella (aplica en claves compuestas).
- **3FN (Tercera Forma Normal):**
Debe cumplir la 2FN y no deben existir dependencias transitivas (ningún atributo no clave puede depender de otro atributo no clave).
- **BCNF (Forma Normal de Boyce-Codd):**
Es una versión más estricta de la 3FN. Exige que para cualquier dependencia funcional $X \rightarrow Y$, X debe ser siempre una superclave (clave candidata). Elimina anomalías sutiles cuando una tabla tiene múltiples claves candidatas compuestas que se solapan.

Problemas que evita la normalización:

- **Anomalías de Inserción:** No poder registrar un dato sin inventar o dejar en nulo otros datos obligatorios no relacionados.
- **Anomalías de Actualización:** Tener el nombre de una ciudad repetido en 10.000 filas; si cambia de nombre y no se actualizan todas, la base de datos queda inconsistente.
- **Anomalías de Borrado:** Al eliminar el último registro de un cliente, borrar accidentalmente la información histórica de una sucursal porque compartían la misma fila.

¿Cuándo se acepta una desnormalización controlada?

Desnormalizar significa introducir redundancia de datos a propósito sacrificando algo de espacio e integridad estricta para priorizar la velocidad de lectura.

Se acepta en escenarios de **alta concurrencia de lectura y analítica**:

- **Optimización de consultas pesadas (Lecturas intensivas):** En un e-commerce, en lugar de hacer un JOIN entre 6 tablas para mostrar el total acumulado y nombre del cliente en una orden, se guardan esos campos directamente en la tabla Pedidos para responder en tiempo récord.
- **Almacenes de datos y BI (Data Warehouses / Esquemas en Estrella):** En sistemas de reportes y tableros donde solo se lee información histórica, la velocidad de consulta masiva es más importante que la normalización relacional.

9) Evitar trabajo repetido y acercar contenido

- a) **Elegí una consulta SQL e implementá, en cualquier lenguaje o pseudocódigo una caché en memoria con cache-aside. Incluí clave, hit/miss, TTL e invalidación y explica brevemente sus limitaciones.**

```

// Memoria caché compartida (ej. Redis o diccionario en memoria)
// Estructura: clave -> { valor: datos, expira_en: marca_de_tiempo }
almacenCache = DICCIONARIO_VACIO

CONSTANTE TTL_SEGUNDOS = 300 // 5 minutos de tiempo de vida (Time To Live)

// 1. Obtener producto aplicando Cache-Aside
FUNCION obtenerProductoPorId(productoId):
    claveCache = "producto:" + productoId
    ahora = tiempoActualEnSegundos()

    // Paso A: Intentar leer desde la caché
    SI almacenCache.existe(claveCache) ENTONCES:
        registro = almacenCache[claveCache]

        // Verificar si el dato aún no venció
        SI ahora < registro.expira_en ENTONCES:
            IMPRIMIR("CACHE HIT: Retornando datos desde memoria")
            RETORNAR registro.valor
        FIN SI
    FIN SI

    // Paso B: CACHE MISS (no existe o venció su TTL)
    IMPRIMIR("CACHE MISS: Consultando base de datos principal")
    producto = BaseDeDatos.ejecutarConsulta("SELECT * FROM productos WHERE id = ?", [productoId])

    SI producto ES NULO ENTONCES:
        RETORNAR NULO
    FIN SI

    // Paso C: Guardar en caché con su TTL para futuras lecturas
    almacenCache[claveCache] = {
        valor: producto,
        expira_en: ahora + TTL_SEGUNDOS
    }

    RETORNAR producto
FIN FUNCION

// 2. Invalidación de caché ante modificaciones
FUNCION actualizarPrecioProducto(productoId, nuevoPrecio):
    // Modificar la base de datos principal
    BaseDeDatos.ejecutarConsulta("UPDATE productos SET precio = ? WHERE id = ?", [nuevoPrecio, productoId])

    // Invalidar / borrar la clave en caché para evitar servir datos viejos
    claveCache = "producto:" + productoId
    almacenCache.eliminar(claveCache)
    IMPRIMIR("CACHE INVALIDADA: Clave eliminada tras actualización")
FIN FUNCION

```

Limitaciones de esta implementación:

- **Datos obsoletos (Stale Data):** Si otro proceso modifica la base de datos directamente sin pasar por la función de invalidación, la caché seguirá entregando datos viejos hasta que venza el TTL.
- **Avalancha de caché (Cache Stampede / Thundering Herd):** Si un producto muy popular vence su TTL en un momento de alto tráfico, cientos de solicitudes concurrentes experimentarán un *cache miss* simultáneo, saturando la base de datos al mismo tiempo.
- **Consumo de memoria RAM:** Guardar demasiados datos sin políticas de expulsión (como *LRU - Least Recently Used*) puede agotar la memoria disponible del servidor.

b) Define CDN e identifica servidor de origen, servidores de borde, contenido habitual y beneficio principal.

Una CDN (*Content Delivery Network*) es una red geográficamente distribuida de servidores que colaboran para entregar contenido web a los usuarios de forma rápida, reduciendo la distancia física entre el cliente y el servidor.

- **Servidor de origen (Origin Server):**
Es el servidor backend o almacenamiento principal donde vive la versión original y autorizada de la aplicación y sus archivos (por ejemplo, tu servidor en Buenos Aires o un bucket de AWS S3).

- **Servidores de borde (*Edge Servers* / Puntos de Presencia PoP):**
Son servidores réplica distribuidos en decenas de ciudades de todo el mundo. Guardan copias en caché del contenido y responden directamente las peticiones del usuario que esté físicamente más cerca.
- **Contenido habitual que distribuye:**
Principalmente archivos estáticos de gran peso o consumo repetitivo: imágenes, videos, hojas de estilo (CSS), scripts de frontend (JS), fuentes tipográficas y documentos PDF.
- **Beneficio principal:**
Reducción drástica de la latencia y descarga de tráfico en el backend: Un usuario en Madrid que consume un sitio alojado en Argentina no espera a que los archivos crucen el océano Atlántico; los descarga en milisegundos desde un servidor de borde ubicado en Madrid, aliviando el consumo de ancho de banda y CPU del servidor de origen.